Fire Detection System for Camera-Monitored Sites

Software Requirements Specification & Project Documentation

Author	Maram	**Course**	Computer Vision Track — Object Detection Module
Institution	DOTPY Academy	**Instructor**	Mohamed Waleed
Model	YOLO11n — transfer learning from COCO	**Date**	August 2026

01 · Project Title

Fire Detection System for Camera-Monitored Sites — a YOLO11-based visual detector that finds fire in video from existing surveillance cameras and raises a confirmed alarm, with a measured false-alarm rate rather than a claimed one.

02 · Problem Statement

Conventional fire detection relies on smoke and heat sensors mounted at ceiling level. A sensor only responds once combustion products physically reach it, which requires the fire to have grown enough to fill the space between the source and the detector. In large or open areas — warehouses, workshops, atriums, yards, plant rooms — that delay can run to minutes, and in outdoor or high-ceiling spaces point sensors are often ineffective or absent altogether.

Meanwhile these sites already have cameras covering the same areas, but nobody watches the footage live — it is reviewed after an incident rather than during one. The camera sees the fire the moment it becomes visible; the information exists and is simply not acted upon.

Object detection is the right tool here rather than classification because the operator's next action depends on *where*: which camera, which part of the frame, which door to send someone through. A model that reports only "this frame contains fire" tells a security officer nothing they can act on. A bounding box does.

03 · Proposed Solution

A frame from a camera is resized with aspect-preserving letterboxing and passed to a YOLO11n detector fine-tuned on fire imagery. The model returns candidate boxes with class labels and confidence scores. Boxes below a calibrated per-class threshold are discarded, overlapping boxes are merged by class-wise non-maximum suppression, and the survivors are matched against detections from previous frames.

A detection becomes an *alarm* only after it persists across several consecutive frames in approximately the same location. The model scores each frame independently and has no memory, so without this step every single-frame false detection becomes an alarm. The output is an alarm event with a timestamp, a location, a confidence value and a saved still image.

04 · Objectives

1. Fine-tune a YOLO11 detector by transfer learning from COCO-pretrained weights, rather than training from scratch. **Met.**
2. Achieve mAP@0.5 ≥ 0.65 on a held-out test split. **Met — 0.710.**
3. Report Precision, Recall, mAP@0.5 and mAP@0.5:0.95 both overall and per class, and interpret each. **Met.**
4. Determine the confidence threshold from measurement on labelled data instead of accepting a default. **Met — calibrated per class.**
5. Measure the false-alarm rate on real event-free footage, not only mAP. **Met — 0 confirmed alarms over 70 s.**
6. Be able to explain which class fails, why, and what would fix it. **Met — documented in §13 and §14.**

05 · Target Users

Role	Decision the system changes
Security / control-room operator	Receives a camera ID, a timestamp and a still image instead of monitoring a wall of screens. Acts on an event rather than scanning for one.
Site safety officer	Gains detection in areas where point sensors are ineffective — high ceilings, open yards, loading bays.

Role	Decision the system changes
Facility / operations manager	Obtains a timestamped, image-backed record of fire events for incident review and insurance.

06 · System Input

- **Format** — video file (MP4/AVI/MOV) or a frame stream from an IP camera.
- **Resolution** — any; internally letterboxed to 640×640. Tested at 1280×720 and 960×540.
- **Sampling** — every 5th frame by default; on 25 fps footage this gives 5 analysed frames per second, adequate for fire, which does not move like a vehicle.
- **Assumptions** — daylight or artificially lit scenes; the fire occupies a reasonable fraction of the frame. Night-time, thermal and heavily obscured footage are out of scope, and §13 documents an observed night-time failure.

07 · System Output

```
Per detection : [x1, y1, x2, y2]  ·  class  ·  confidence 0–1
Per alarm     : timestamp HH:MM:SS · class · peak confidence · frame index · saved JPEG
Per run       : confirmed alarms · alarms per hour · first-alarm latency · frames analysed
Files         : report.json · alarms.csv · annotated MP4
```

The distinction between a *detection* and an *alarm* is deliberate. Detections are per-frame model output; alarms are confirmed events. On the fire clip, 153 raw detections produced 2 alarms.

08 · Dataset

Field	Value
Name	Fire-smoke Detection (fire-smoke-detection-lk8z9), version 1
Source	Roboflow Universe, workspace detection-projet — universe.roboflow.com/detection-projet/fire-smoke-detection-lk8z9/dataset/1
Licence	CC BY 4.0 — free use with attribution. Checked before use.
Classes	nc: 2 — fire, smoke
Format	YOLO, already annotated with bounding boxes. No manual annotation performed.

Split and class balance

Split	Images	Instances (test)	Count
Train	10,589	fire	479 (82.9%)
Validation	1,017	smoke	99 (17.1%)
Test	**521**	**Total**	**578**
Total	**12,127**		

Why this dataset was selected

It is already annotated in YOLO format with a ready train/validation/test split, it is large enough that the nano model is not starved of data, and it consists of genuine fire photographs and video frames rather than staged images. Its licence permits use with attribution.

Weaknesses — named rather than hidden

1 · Severe class imbalance. Fire outnumbers smoke roughly 5:1. This is the direct cause of the weakest result in the project and is analysed in §13.

2 · Augmentation applied before the split. The version page reports 5,068 source images expanded to 12,127. Roboflow augments only the training split, yet 386 of the 521 *test* filenames carry _aug1/_aug2/_aug3 suffixes, and the 521 files trace back to roughly 133 distinct source frames. The publisher therefore uploaded pre-augmented images, which were then split at random — so augmented siblings of the same source frame appear on both sides of the split. **The reported test metrics are consequently optimistic by an unknown margin.**

3 · No background images. Ultralytics reports 0 backgrounds when scanning every split: every image contains at least one object. The model was never shown an empty scene during training, so its behaviour on ordinary event-free footage is not constrained by the training data at all. This motivated the false-alarm measurement in §12.

4 · Daylight bias. The imagery is predominantly daylight or bright-flame footage, which produced the night-time failure documented in §13.

09 · Model

Item	Value
Architecture	YOLO11 nano — 101 fused layers, 2,582,542 parameters, 6.4 GFLOPs
Transfer learning	Initialised from yolo11n.pt, pretrained on COCO
Run 1 (selected)	epochs=20, imgsz=640, batch=32, optimizer=auto (SGD, lr0 0.01) — ~57 min on a Tesla T4
Run 2 (rejected)	epochs=50, imgsz=640, batch=16, optimizer=AdamW, lr0=0.001, mosaic=0.5, close_mosaic=10
Framework	Ultralytics 8.4.121, PyTorch 2.11.0 + CUDA
Deployment format	ONNX opset 20, dynamic axes, BatchNorm fused, NMS external

Why nano

The nano checkpoint trains in under an hour on a free notebook GPU, which left time for evaluation and analysis rather than consuming the whole project in training. It is also fast enough to run the video demo unchanged: 4.9 ms inference per frame on a T4. COCO pretraining supplied general edge, texture and shape features, so only the fire distinction had to be learned — which is why roughly ten thousand images and one free GPU session were sufficient.

What transfer learning contributed. Training this architecture from random initialisation on 10,589 images would not converge to anything useful. The pretrained backbone already encodes features shared by all natural images; fine-tuning re-purposes the later layers and the detection head for two new classes.

10 · System Workflow

```
Camera / video file
  →  letterbox to 640×640, grey 114 padding (preserves aspect ratio)
  →  YOLO11n ONNX inference
  →  per-class confidence threshold — fire 0.20 (calibrated)
  →  class-wise non-maximum suppression, IoU 0.45
  →  size bands — reject specks, route whole-frame boxes to a slower path
  →  temporal confirmation — 5 detections within 8 frames at the same location
  →  duplicate suppression — one alarm per event
Alarm : timestamp · box · confidence · saved frame
```

Letterboxing matters more than it appears: the model was trained on letterboxed input, so resizing a 16:9 frame directly to a square distorts objects by 1.78× and presents geometry the model never saw.

11 · Evaluation

All figures below are on the **test split** — 521 images, 578 instances, never used in training or validation — measured with Ultralytics 8.4.121 on a Tesla T4 at imgsz=640.

Selected model — run 1, 20 epochs

Class	Precision	Recall	mAP@0.5	mAP@0.5:0.95
All	**0.739**	**0.722**	**0.710**	**0.336**
fire	0.869	0.889	0.914	0.482
smoke	0.610	0.556	0.507	0.190

What each number means for this problem

- **Precision 0.739** — of every ten boxes the model draws, about seven are correct. For an alarm system this is the "can it be trusted when it speaks" figure. It is carried by fire (0.869) and dragged down by smoke (0.610).

- **Recall 0.722** — it finds roughly seven of every ten objects actually present. Fire recall of 0.889 is acceptable; smoke recall of 0.556 means nearly half of all smoke goes unseen.
- **mAP@0.5 = 0.710** — the summary score at a forgiving IoU threshold. The per-class split shows it is carried by one easy class: fire 0.914, smoke 0.507. The headline number alone would hide that.
- **mAP@0.5:0.95 = 0.336** — averaged across stricter IoU thresholds, so it rewards tightly placed boxes. It is always lower, and for fire this is expected: flame has no crisp edge, so a "correct" box is inherently approximate.

> **External reference point.** The dataset publisher trained their own model on the same data and reports mAP@0.5 68.1%, precision 72.0%, recall 67.6%. This project's run 1 reaches **71.0% / 73.9% / 72.2%** — higher on all three, on the same dataset.

Model comparison — the shorter run won

Two runs were trained and scored under identical conditions on the same held-out test split.

Metric	Run 1 — 20 epochs	Run 2 — 50 epochs, AdamW
mAP@0.5	**0.710**	**0.686**
mAP@0.5:0.95	0.336	0.333
Precision	0.739	0.763
Recall	0.722	0.651
fire — mAP@0.5	0.914	0.912
smoke — recall	**0.556**	**0.414**
smoke — mAP@0.5	0.507	0.460

Run 2 used 2.5× the epochs, a different optimiser, a tuned learning rate and reduced mosaic augmentation — and scored *lower*. Fire was untouched (0.914 vs 0.912); the entire loss landed on smoke, whose recall fell 26% in relative terms. Run 2's own training curves explain it: val/dfl_loss reached its minimum around epoch 14 and then climbed steadily to epoch 50, while training loss kept falling. The model was memorising. Longer training was not neutral here — it was harmful.

Threshold calibration

The confidence threshold is a decision, not a default. Every test image was scored at thresholds from 0.05 to 0.95.

Class	F1-opt thr	F1	Precision	Recall	Thr for P ≥ 0.90	Recall there
fire	0.43	0.898	0.935	0.864	0.20	0.881
smoke	0.16	0.525	0.525	0.525	0.86	**0.020**

Fire is usable. It reaches 90% precision at a threshold as low as 0.20 while retaining 0.881 recall — the model is both confident and correct about fire.

Smoke is not. At its best operating point, half the smoke alarms are false and half the real smoke is missed. Pushing precision to 90% requires a threshold of 0.86, at which recall collapses to 0.020 — **2 of the 99 smoke instances**. There is no point on the curve where smoke detection is deployable. This is a data limitation, not a tuning problem, and no threshold choice can repair it.

Speed — measured, not claimed

Environment	Configuration	Per frame	FPS
Tesla T4 (GPU)	YOLO11n, 640×640, inference only	4.9 ms	~204
Tesla T4 (GPU)	full pipeline incl. pre/post-processing	14.7 ms	~68
Colab CPU	video pipeline, 1280×720, ONNX Runtime	135 ms	7.4

On GPU the system is comfortably real-time. On CPU at 7.4 fps it is **not** real-time and is reported as such; with frame sampling every 5th frame it still keeps pace with a 25 fps source, but the honest description is "fast enough for the sampling rate used", not "real-time".

12 · Results

Unseen test images

Twelve images drawn from the held-out test split with a fixed random seed, at conf=0.25. Eleven produced correct fire detections at 0.47–0.85 confidence. One produced no detection at all. **Only one of the twelve produced a smoke detection**, although smoke is visible in most fire imagery — a visual confirmation of the 0.556 recall figure.

External images — outside the dataset entirely

Four images downloaded from the web with no relation to the dataset, at conf=0.25:

Image	Ground truth	Detection	Verdict
Clear Fire	fire	fire 0.50, fire 0.37	**correct**
Smoke	smoke, no visible flame	NONE	**missed**
Sunset	no fire, no smoke	NONE	**correct**
Fog	no fire, no smoke	NONE	**correct**

Zero false positives on the deceptive backgrounds. Orange sunset light and dense fog are the classic false-alarm triggers for a fire detector, and neither produced a detection. Given that the training set contains no background images, this was not guaranteed.

This corrected an earlier hypothesis of my own: probing the model with flat synthetic colour fields produced high-confidence detections, suggesting a colour shortcut — "orange means fire". Real photographs did not reproduce that behaviour, so the effect is limited to degenerate out-of-distribution input. The hypothesis was discarded on the evidence.

Fire generalises, with a narrower margin. The external fire image was detected at 0.50 and 0.37, against the 0.75–0.85 typical of test-split images, and the second box falls below the calibrated 0.43 threshold. This is consistent with a distribution gap: the dataset draws on a limited set of sources, and an arbitrary web image sits outside that.

Video demonstration

Two clips through the identical pipeline at the calibrated threshold, sampling every 5th frame.

Measure	Office · 70 s · no fire	Fire clip · 26 s
Frames analysed	350	158
Raw detections per frame	0.020	0.968
Confirmed alarms	**0**	**2**
First alarm at	—	2.0 s
Alarms/hour without temporal confirmation	360	21,002

> **The two clips are only meaningful together.** A broken model would also produce zero alarms on the office clip. This one produces zero there *and* alarms within two seconds on fire — a 48× difference in raw detection rate between the two scenes. That is discrimination, not silence.

The office result quantifies the confirmation step: the same footage would have generated **360 alarms per hour** with per-frame alarming, and **none** with confirmation. Its seven raw detections were isolated flickers; none persisted for five frames in one location.

Sample-size caveat. Seventy seconds of one daylight scene is a small sample. Zero alarms over that span is consistent with any true rate below roughly 50 per hour. This is an indication, not an established false-alarm rate.

13 · Challenges

1 · The class I cared about is the class that failed

Smoke normally precedes visible flame by minutes, so it is the class that would provide early warning — and it is the weakest result in the project. Recall 0.556, precision 0.610, and no usable operating point at any threshold. Four independent lines of evidence converge: the calibration curve, the per-class metrics, eleven of twelve unseen fire images with no smoke box, and an external smoke image with no detection at all. The cause is the 5:1 imbalance combined with the intrinsic difficulty of the class

— fire has a consistent signature (bright, saturated, high contrast) while smoke is diffuse, low-contrast and takes its colour from the background.

Decision taken: the smoke class is disabled in the demonstration rather than shipped at 52% precision. Its metrics are still reported in full. Shipping a class that is wrong half the time would poison the alarm channel that fire detection depends on.

2 · More training made the model worse

My assumption was that the first run at 20 epochs was under-trained. Run 2 used 50 epochs, AdamW, a lower learning rate and reduced mosaic — and scored lower on the test split, losing 26% of smoke recall. Validation-set numbers had suggested a slight improvement; only the held-out test split revealed the truth. The lesson is that validation metrics were not sufficient to choose between the two models.

3 · I lost track of which model produced which numbers

Weights were downloaded, re-uploaded and moved between sessions until the files were named best.pt, best (1).pt and best (1) (1).pt. At that point the exported ONNX could not be attributed to a training run, and the metrics in hand did not necessarily describe the model in hand.

Fix: every evaluation and export now records a SHA-256 hash of the file it operated on, written into a manifest alongside the metrics and thresholds. The shipped model is identified as weights ff6a0d6b… → ONNX 99ea3776… → mAP@0.5 0.710. This was the single most useful process change in the project.

4 · A night-time scene produced no detection at all

One unseen test image — a dark night fire scene with scattered dull orange glow rather than a bright flame mass — returned nothing at conf=0.25. The training data is overwhelmingly daylight or bright-flame footage, so the model has effectively learned that fire means *a bright, high-saturation, high-contrast region*. That cue disappears at night. For a fire alarm this is the dangerous failure direction: a missed fire at the hour when a building is least likely to be occupied by someone who would notice it.

5 · mAP alone does not tell you if the system is deployable

mAP is measured on images that all contain objects. It says nothing about behaviour on ordinary footage where nothing is happening — which is what a camera sees 99.99% of the time. The 360-alarms-per-hour figure on an event-free office clip does not appear in any mAP number, and is the figure that would actually decide whether an operator keeps the system switched on.

14 · Future Improvements

Each is tied to a weakness observed in this project, in order of expected return.

7. **Add background images — 1,500–2,500, no annotation required.** The training set contains zero. Ordinary scenes with no fire — sunsets, sodium street lighting, fog, steam, orange safety clothing — would give the model explicit negative evidence. This is the cheapest fix, needs no labelling, and directly addresses §13.5.
8. **Add 800–1,000 annotated smoke instances.** The heaviest task and the highest value: it is what would turn a fire detector into an early-warning system. It directly targets the weakest metric in §11.
9. **Add night and low-light footage.** Directly addresses the failure in §13.4. Brightness and gamma augmentation would be the cheapest first test, but genuine night imagery is what is actually missing.
10. **Re-split the dataset at source-clip level.** All augmented siblings of one frame must sit in one split. Metrics would fall — that is the point, since the current figures are inflated by the leakage documented in §08.
11. **Capture an hour of real event-free site footage.** Seventy seconds is not enough to establish a false-alarm rate. This is cheap and fast and would convert §12's indication into a measured figure.
12. **Compare against YOLO11s.** There is ample compute headroom at 4.9 ms per frame, and the extra capacity would most likely benefit smoke, the visually harder class.